

PATENT SPECIFICATION
UNDER 35 U.S.C. § 111(b) (PROVISIONAL APPLICATION)

**SYSTEM, METHOD, AND APPARATUS FOR ZERO-COPY IN-PLACE
COMPACTION AND ASSOCIATIVE ROUTING OF DYNAMIC KEY-VALUE
(KV) CACHE TENSORS IN DEEP LEARNING ACCELERATORS**

INVENTORS:

1. Dr. Ahmet Emre Çetin

Citizenship: Republic of Turkey

2. Mr. Eren Çetin

Citizenship: Republic of Turkey

CROSS-REFERENCE TO RELATED APPLICATIONS

[0001] This application claims the benefit of priority under 35 U.S.C. § 119(e) of provisional application filing parameters established on the filing date hereof.

**STATEMENT REGARDING FEDERALLY SPONSORED RESEARCH OR
DEVELOPMENT**

[0002] Not Applicable. No federal government funds or grants were used in the conception or development of this invention.

FIELD OF THE INVENTION

[0003] The present invention relates generally to deep learning hardware accelerators, parallel processors, and memory architectures. More specifically, the present invention relates to dynamic memory allocation, Key-Value (KV) cache pruning, zero-copy in-place memory compaction, and content-addressable memory routing for generative transformer models and Large Language Model (LLM) inference engines.

BACKGROUND OF THE INVENTION

[0004] In modern autoregressive deep neural networks—such as transformer-based Large Language Models (LLMs)—the generation of each sequential token requires evaluating an attention mechanism over the entire historical context. To avoid recalculating past context states at each generation step, intermediate Key (K) and Value (V) activation states are stored in an accelerator memory (such as High Bandwidth Memory [HBM], GDDR, or SRAM) as a dynamic data structure known as the 'KV-Cache.'

[0005] As the context window length increases (e.g., beyond 32k, 128k, or 1M tokens), the linear and quadratic memory growth of the KV-cache creates severe computational and memory capacity bottlenecks (the 'Memory Wall'). To mitigate this, dynamic context eviction algorithms discard less important tokens to bound the cache within a fixed budget.

[0006] However, conventional eviction techniques exhibit severe hardware trade-offs: (1) Dynamic Memory Allocation Overhead: Discarding non-contiguous tokens leaves sparse gaps in memory buffers, requiring intermediate system calls (cudaMalloc) and auxiliary copies ($O(N)$ auxiliary buffers) to pack active tokens into a contiguous block; (2) Page-Table Indirection Latency: Paged memory techniques allocate non-contiguous physical pages via virtual lookup tables, preventing direct contiguous Tensor Core memory streaming; (3) Inefficient Search Routing: Standard Von Neumann memory architectures require sequential address lookup passes, lacking hardware-level associative content-addressing.

[0007] Therefore, there exists an acute technical need in the art for an apparatus, system, and method capable of dynamically evicting and compacting KV-cache tensors in-place (zero-copy) with strictly bounded $O(1)$ auxiliary storage, maintaining physical memory contiguity, and utilizing single-pass deterministic associative convergence without pipeline stalls.

BRIEF SUMMARY OF THE INVENTION

[0008] The present invention solves the aforementioned technical deficiencies by providing a system, hardware architecture, and kernel execution method that executes in-place zero-copy tensor compaction of dynamic KV-cache structures using a hardware-level virtual Content-Addressable Memory (CAM) mapping combined with an idempotent cyclic permutation mechanism.

[0009] In one embodiment, an integrated memory management controller computes retention priority scores for stored token tensors. These scores are mapped via an associative characteristic function to target contiguous destination slots. The mapping enforces an algebraic idempotence condition ($f(f(x)) = f(x)$), ensuring that active tokens placed in stabilized target attractor basins form mathematical fixed points that do not oscillate across consecutive compaction epochs.

[0010] In another embodiment, the controller partitions the permutation space into a set of mutually disjoint permutation cycles. The controller determines cycle leaders using an $O(1)$ scalar traversal verification without allocating auxiliary boolean bitmasks or marking arrays.

[0011] An in-place cyclic permutation kernel shifts tensor blocks along the disjoint cycles using a single temporary register buffer. Tokens marked for retention are packed contiguously at the front of the physical memory allocation, while tokens marked for eviction are permuted to the tail boundary. The controller then resets the active buffer boundary pointer, releasing or overwriting evicted memory without issuing host operating system allocation commands or auxiliary buffer allocations.

BRIEF DESCRIPTION OF THE SEVERAL VIEWS OF THE DRAWINGS

[0012] FIG. 1 is a high-level system hardware architecture diagram illustrating a deep learning accelerator comprising a High Bandwidth Memory (HBM), an integrated Memory Management Controller, and parallel compute units in accordance with an embodiment of the present invention.

[0013] FIG. 2 is a comparative state transition diagram showing physical memory layouts before and after in-place compaction, depicting the displacement of tensor blocks along disjoint permutation cycles into an active contiguous region and a truncated region.

[0014] FIG. 3 is a logical flowchart illustrating the execution steps of the in-place cycle-leader traversal and cyclic swap algorithm executed with $O(1)$ auxiliary register memory.

DETAILED DESCRIPTION OF PREFERRED EMBODIMENTS

[0015] Referring to FIG. 1, the system encompasses a deep neural network accelerator (100) (e.g., a GPU, NPU, or domain-specific ASIC). The accelerator (100) includes a primary accelerator memory (102) storing a physically contiguous KV-cache tensor buffer (104) having N allocation slots. The accelerator further includes a plurality of Streaming Multiprocessors / Parallel Compute Units (140) and an integrated Memory Management Controller (110).

[0016] The Memory Management Controller (110) comprises: (1) An Associative Scoring & Characteristic Mapping Unit (112) configured to track cumulative attention mass metrics (S_i) across sliding windows and project them into target buffer tiers; (2) An Idempotent Projection Engine (114) configured to convert target allocations into an idempotent index map satisfying $f(f(x)) = f(x)$; (3) An In-Place Disjoint Cycle-Leader Execution Kernel (116) configured to execute cyclic permutations directly across the contiguous buffer without scratchpad memory.

[0017] Referring to FIG. 2, an exemplary compaction scenario is illustrated. Prior to compaction, $N=6$ token blocks (T_0 to T_5) are stored in indices 0 through 5. Tokens T_0 , T_2 , and T_4 exhibit high utility, while tokens T_1 , T_3 , and T_5 exhibit low utility.

[0018] The Associative Scoring Unit generates the map: $f(0)=0$ (Fixed Point), $f(1)=3$, $f(2)=1$, $f(3)=5$, $f(4)=2$, $f(5)=4$. The cycle resolution decomposes into fixed point (0) and a single disjoint cycle: (1 -> 3 -> 5 -> 4 -> 2 -> 1).

[0019] The Execution Kernel loads Block 1 into a temporary register (Temp_Reg). Block 2 copies to Block 1; Block 4 copies to Block 2; Block 5 copies to Block 4; Block 3 copies to Block 5; and Temp_Reg writes into Block 3. Following compaction, indices [0, 1, 2] contain active contiguous tokens, while indices [3, 4, 5] contain evicted tokens.

[0020] Referring to FIG. 3, at step 302 compaction begins. At step 304 map f is generated. At step 306 index i is set to 0. Steps 308-312 check termination and fixed points. Steps 314-322 check if index i is the minimal cycle leader ($\text{curr} < i$). If confirmed, steps 324-326 perform in-place cyclic shifts using Temp_Reg.

CLAIMS

What is claimed is:

[0021] Claim 1. A computer-implemented method for zero-copy in-place memory compaction and associative routing of dynamic Key-Value (KV) cache tensors within an accelerator memory of a deep learning processor, the method comprising: (a) monitoring attention retention metrics associated with N tensor blocks stored within a contiguous region of accelerator memory; (b) generating a target permutation index vector $f(i)$ defining an idempotent projection satisfying $f(f(x)) = f(x)$, establishing invariant fixed points; (c) identifying a set of cycle leaders

corresponding to mutually disjoint permutation cycles using strictly bounded $O(1)$ scalar auxiliary register storage without external boolean bitmasks; (d) executing an in-place cyclic permutation of tensor blocks along each disjoint permutation cycle using a singular temporary register buffer, shifting memory payloads directly into destination addresses; and (e) truncating an active allocation boundary pointer to retain a contiguous block of active tensors and release evicted memory without operating system dynamic allocation commands.

[0022] Claim 2. A deep learning computing system comprising: a primary accelerator memory storing a plurality of Key-Value (KV) cache tensor blocks within a contiguous buffer; an execution engine comprising a plurality of parallel compute cores; and a memory management controller operatively coupled to the memory and execution engine, configured to: compute utility priority metrics for each stored tensor block; construct an idempotent associative map translating utility priority metrics into contiguous target index addresses; dispatch an in-place tensor compaction kernel to permute tensor blocks along disjoint permutation cycles via minimal-index cycle leader validation using only $O(1)$ temporary register memory; and output a physically contiguous, compacted sequence without auxiliary scratchpad memory buffers.

[0023] Claim 3. The system of Claim 2, wherein the memory management controller comprises dedicated fixed-function hardware circuitry embedded within a memory crossbar interface of the primary accelerator memory, configured to calculate cycle leader traversals concurrently with tensor core matrix multiplication operations.